

NeuFlow v3

A queryable, uncertainty-aware flow decoder
for edge robotics

The decoder works and is measured. It costs 2.6 percent of mean accuracy, buys three capabilities the baseline cannot express, and has one diagnosed limitation that is not yet closed.

Two of the three objectives I set are not met.

Three capabilities were not on the list

Objective set at the start	Verdict	Evidence
Better accuracy than v2	not met	2.384 against 2.324; like-for-like 7.6% behind dense 10% slower; repeat query 27x cheaper every figure is from a RTX 4060 laptop
Less compute than v2	partly	
Runs on edge devices	unproven	

DELIVERED, AND MEASURED

Queryable output

flow at any continuous coordinate;
sparse reproduces dense exactly

Cheap repeat access

1.25 ms per further query batch
against a cached frame

Calibrated confidence

a per-query error estimate,
monotonic against real error

The change is confined to the last stage, and the front end is verified unchanged

Phase 1, once per frame pair. v2 exactly as published, up to its 1/8 coarse flow. 33 ms, and the result is cached.

Phase 2, once per query batch. A coordinate goes in, a convex blend of the nine neighbouring coarse-flow values plus a bilinear sample comes out. 1.25 ms, and cost scales with the number of queries rather than with image area.

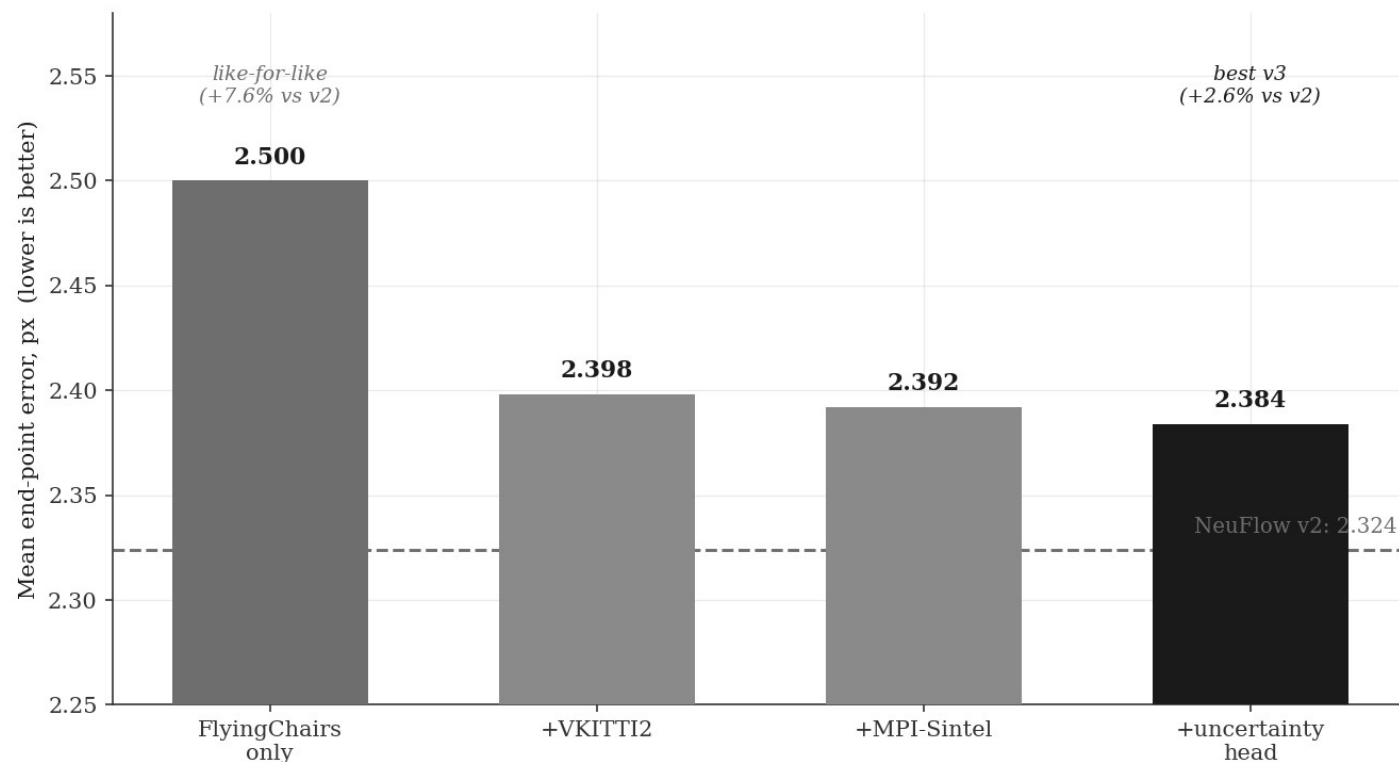
The control. All 137 tensors shared with v2 are verified bit-identical per checkpoint, and both evaluation scenes are held out of every training set.

Because the blend is convex, the decoder cannot predict motion its inputs do not locally support.

Zero-initialised, the decoder is exactly bilinear upsampling.

Because Phase 1 is cached, a second question about a frame already processed costs 1.25 ms instead of 33.3. That is the property the rest of this deck turns on.

The price is 2.6% of mean error, from a model 13% smaller



THE COST, STATED FIRST

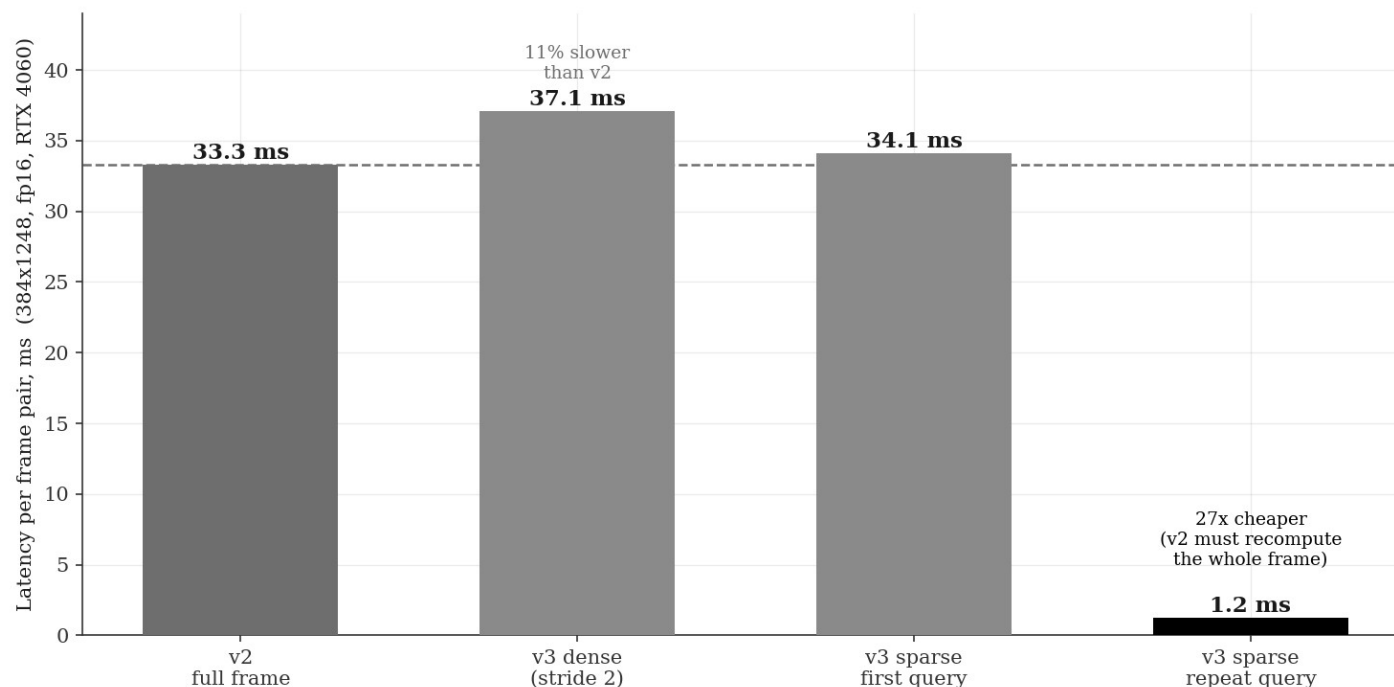
Every configuration sits above v2. The best is 2.384 against 2.324.

Like-for-like, trained on FlyingChairs alone as v2 was trained on FlyingThings alone: 2.5, or 7.6% behind.

7.83 M parameters against 9.03 M.

2.384
against 2.324

State reuse is the win: a repeat query costs 1.25 ms against 33.3 ms



WHICH CLAIM I AM MAKING

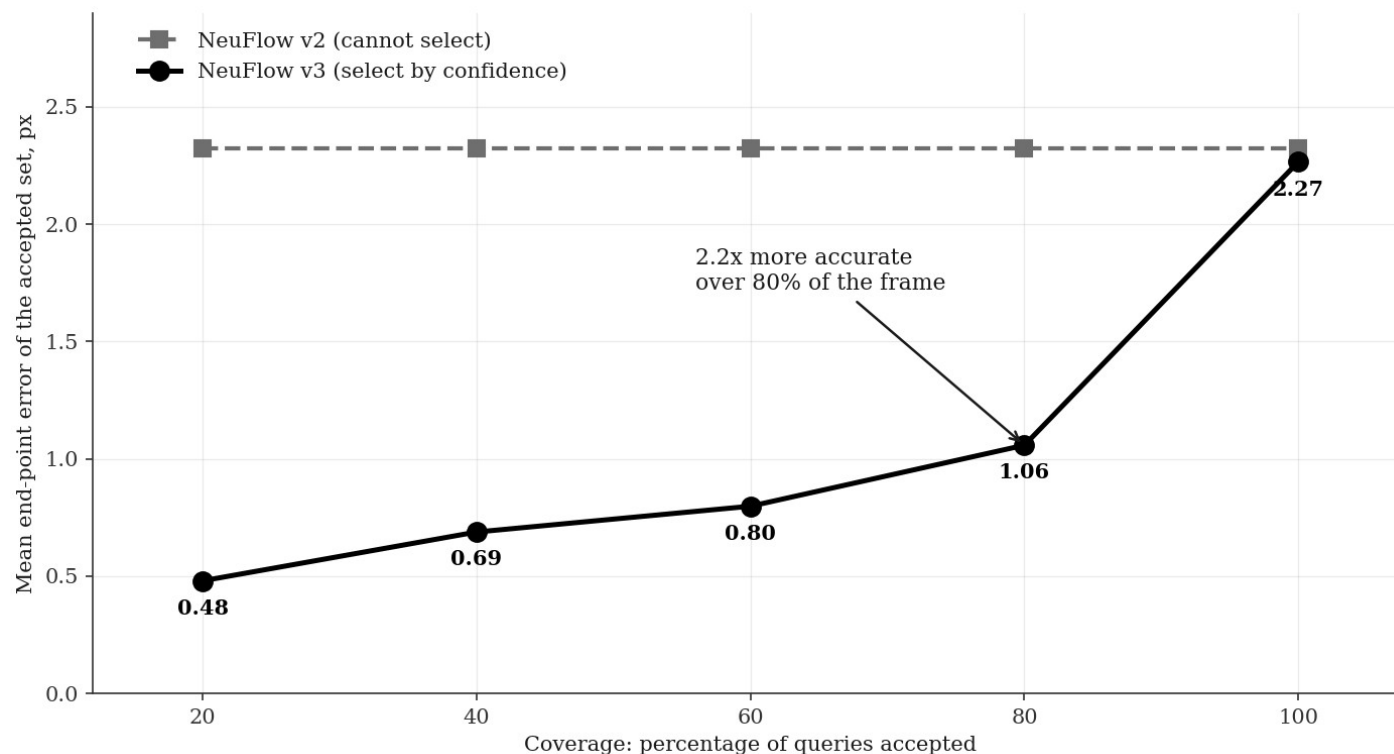
Dense output is 10% slower than v2. That mode is not what the decoder is for.

A first sparse query is level with v2: 96% of the cost is the coarse pass, which both pay and which cannot be skipped.

The win is the second question about the same frame. v2 keeps no state, so it recomputes everything.

27x on repeat access

Confidence is the capability v2 cannot express, and it buys accuracy back



CALIBRATION BEHIND THE CURVE

Real error rises monotonically across five bins, 0.48 px to 7.10 px, over 2,348,000 queries.

Pearson $r = 0.318$: informative rather than tight, and usable for weighting or rejecting matches.

v2 emits flow alone, so it has one operating point where v3 has a curve.

**1.058 px over 80%
of the frame**

Flowing 14% of the frame costs 0.034 px and runs 4.4x faster

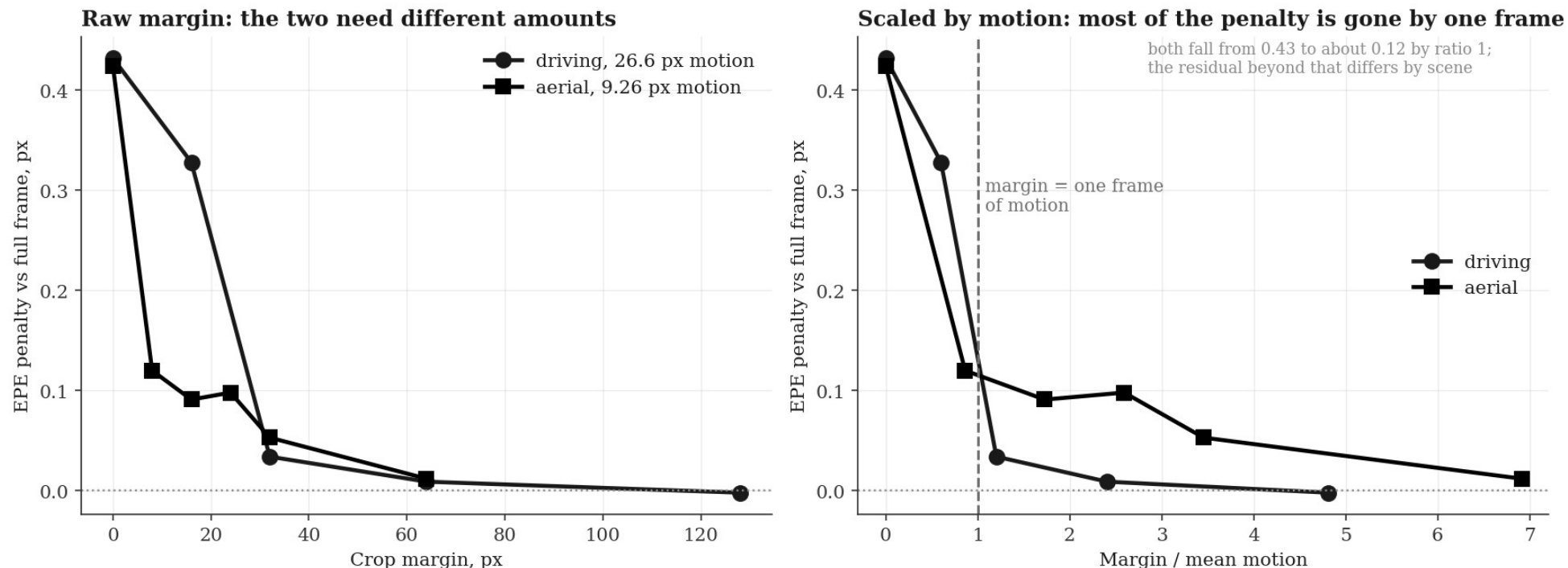
Region processed	Area	Latency	Speedup	EPE in region
Full frame	100%	33.3 ms	1.0x	0.657
Region, no margin	7.9%	7.8 ms	4.3x	1.089
Region + 32 px margin	13.8%	7.6 ms	4.4x	0.691
Region + 64 px margin	20.1%	8.1 ms	4.1x	0.667
Region + 128 px margin	34.2%	9.9 ms	3.4x	0.655

Design rule

margin $\approx$ expected motion
 $\approx$ speed / frame rate

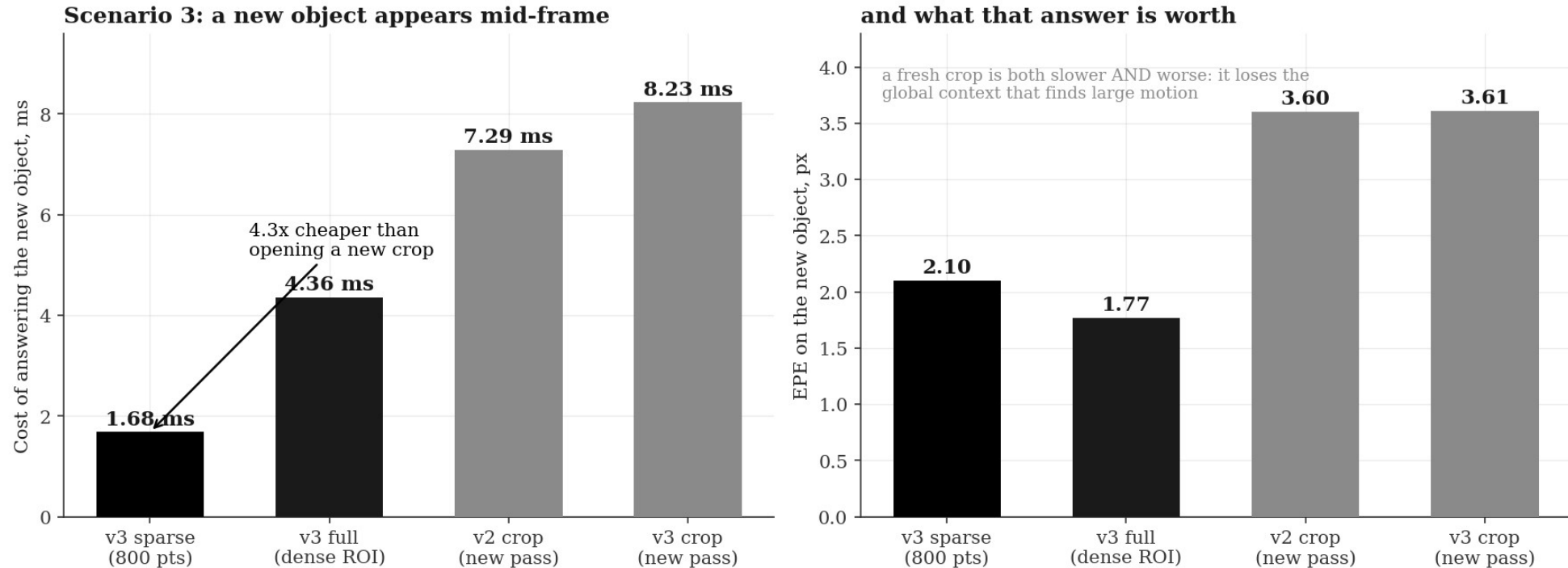
The margin is not optional. Without one, error rises 65 percent and a quarter of large-motion pixels fail outright, because global matching loses the context it uses to locate distant correspondences.

The margin needed is set by motion, not by image size



Driving averages 26.6 px of motion and needs about 32 px of margin. Aerial averages 9.26 px and needs about 8 px. Both shed most of the penalty by one frame of motion, so a margin can be sized from speed and frame rate before anything is run.

A region found mid-frame costs 1.7 ms, against 7.3 ms for a fresh crop



The cached coarse state answers a question the platform did not have when the frame arrived. A cropped pipeline must run an entire new pass, and is less accurate doing it. Two disjoint crops cost more than one full frame: crop once, or not at all.

On a domain neither model trained for, v3 leads at all seven margin settings

Crop margin	NeuFlow v2	NeuFlow v3
full frame	0.781	0.777
0 px	1.205	1.176
8 px	0.901	0.881
16 px	0.872	0.865
24 px	0.879	0.854
32 px	0.834	0.814
64 px	0.793	0.778

HOW MUCH IT COUNTS

The only unconfounded comparison I have. Every other table is on a domain v3 trained on and v2 did not.

Consistent rather than large: 0.004 to 0.029 px, over 40 pairs and one seed.

v3 is still worse on large-motion failures, 7.2% against 6.5%.

One diagnosed cause, three experiments converging on it, not yet fixed

The cause. The decoder's finest input is at 1/8 resolution, so the evidence available to it barely varies inside an 8x8 cell. The upsampler it replaces reads the full-resolution frame directly.

Three independent nulls. A Fourier positional encoding, sub-pixel querying, and reduced-resolution querying all returned null, and all three are explained by that one mechanism.

What it also explains. Querying between pixels is presently closer to interpolation than inference. The interface is exact: the extra information recoverable is

This is the only candidate that is specific to v3. The baseline's upsampler already reads full resolution, so nothing here transfers to it.

Proposed remedy

A full-resolution stem: a cheap stride-8 convolution over the full-resolution image, fed to the decoder. Roughly 1 to 2 ms.

What I would do next, in priority order, with what each costs

Next experiment	Why it is next	Cost
Full-resolution stem	the direct attack on the one diagnosed cause	half a day, 5 h train
Repeat uncertainty run, new seed	the 2.392 to 2.384 gain is inside checkpoint noise	5 h train
Jetson measurement	the word edge is currently a design target, not a result	hardware dependent
Spring 4K evaluation	the one test the baseline structurally cannot take	1 h, script exists
Tiled refinement	roughly 2x, but it would benefit the baseline equally	not built

Only the first is a claim about this architecture. The second decides whether the uncertainty head's accuracy gain is real. The third decides whether the edge premise can be stated at all.

Four places where your judgement would change what I do

Is selective prediction a fair reframing of the accuracy result, or does it need a different baseline?

It is the strongest thing I have and the thing I am least certain is being presented correctly.

Full-resolution stem first, or embedded measurement first?

The stem closes the scientific gap. The measurement closes the claim in the title. There is not time for both to be leisurely.

Is one seed per configuration acceptable for the write-up?

It is what I have. It means nothing below about 0.05 px is resolved, and I have said so wherever a difference is smaller than that.

Is the aerial result worth foregrounding, given it is 40 pairs?

It is the only comparison in the study without a training-domain confound, but it is also the smallest sample.

Full result table

Model	Training data	EPE	1 px	Params
NeuFlow v2	FlyingThings	2.324	77.63	9.03 M
NeuFlow v3	FlyingChairs	2.500	72.81	7.83 M
NeuFlow v3	+ VKITTI 2	2.398	75.74	7.83 M
NeuFlow v3	+ MPI-Sintel	2.392	75.83	7.83 M
NeuFlow v3	+ uncertainty head	2.384	76.13	7.83 M

COMPUTE, SAME DEVICE THROUGHOUT

Mode	LatencyNote
v2, full frame	33.3 msits only mode
v3 sparse, first query on a new pair	34.1 msequivalent to v2
v3 sparse, repeat query on a cached pair	1.25 ms27x cheaper
v3 dense output, stride 2	37.1 msnot the intended mode